

AdventureWorks Analytics Pipeline (Day 5 Hackathon Submission)

Database Overview

The underlying data source is the **AdventureWorks** sample database, simulating a global bicycle manufacturing and sales company. The raw operational database is highly normalized (OLTP) and scattered across five domains: Sales, Production, Purchasing, HumanResources, and Person. While optimized for fast daily transactions, this raw structure is complex and slow for business intelligence reporting.

Analytics Architecture

To bridge the gap between operational data and executive reporting, a **4-Layer Chained Analytical Pipeline** was engineered inside PostgreSQL.

- **Layer 1: Core Dimensions** – Flattens complex, multi-table relationships into single reusable lookups (e.g., merging Product, Category, and Subcategory).
- **Layer 2: Core Facts** – Merges transaction headers and detail lines, pre-calculating essential line-item mathematics (e.g., `linetotal`, `stockedqty`).
- **Layer 3: Analytical Aggregations** – Reusable intermediate views summarizing metrics by business entity (e.g., lifetime value by customer, overall performance by employee).
- **Layer 4: Executive KPI Datasets** – The final presentation layer. Fully dashboard-ready datasets built exclusively for direct querying by the Python visualization suite.

Intermediate Tables Created

19 distinct analytical views were created within the `analytics` schema to prevent querying the operational tables repeatedly:

- **Dimensions (6):** `dim_product`, `dim_customer`, `dim_territory`, `dim_employee`, `dim_vendor`, `dim_date`
- **Facts (3):** `fact_sales`, `fact_purchases`, `fact_inventory`
- **Metrics (4):** `customer_metrics`, `product_metrics`, `employee_performance`, `regional_analysis`
- **Executive KPIs (6):** `kpi_sales`, `kpi_customers`, `kpi_products`, `kpi_employees`, `kpi_territories`, `kpi_inventory`

SQL Design Decisions

- **Chained Dependencies:** The pipeline explicitly leverages the outputs of lower layers. Layer 4 queries Layer 3, never the raw tables. This ensures metrics are calculated once and reused everywhere.
- **Advanced SQL Utilization:**
 - **Window Functions:** `LAG()` is used in `kpi_sales` to calculate Month-over-Month (MoM) revenue growth.
 - **Ranking Functions:** `DENSE_RANK()` and `RANK()` dynamically generate Top 10

- leaderboards for products, territories, and employees.
- **Conditional Aggregation:** Used heavily to segment online vs. offline revenue directly within aggregation blocks.
- **Chained CTEs:** Employed in `kpi_sales` and `kpi_territories` to structure multi-step mathematical calculations cleanly.
- **CASE WHEN:** Extensively utilized for dynamic business categorization, such as bucketing customer lifetime value segments and triggering inventory health alerts.

Challenges Faced

1. **Computed Columns Missing in Postgres:** During the initial CSV import, computed SQL Server columns like `LineTotal` and `StockedQty` were dropped by the `install.sql` script. This was resolved by manually reconstructing the mathematical logic (`ReceivedQty - RejectedQty`) directly within the Layer 2 `fact_sales` and `fact_purchases` views.
2. **Visualizing Deprecated Seaborn Palettes:** Encountered `FutureWarning` errors in the Jupyter Notebook due to Seaborn's recent update requiring a hue assignment for customized bar plots. This was quickly resolved by explicitly mapping the `x` variable to hue while disabling the legend.

Assumptions Made

1. **Currency:** All monetary values are assumed to be standard USD.
2. **Date Boundaries:** The `dim_date` dimension was explicitly generated to cover dates from `2000-01-01` through `2030-12-31` to safely encompass all potential transactional data in the provided datasets without leaving gaps.
3. **Data Integrity:** We assumed that the imported CSV data contained complete header-to-detail relationships (no orphaned records during INNER JOINS).